NestIQ Pro

Smart rental management platform (Landlord + Tenant portal)

- Project: NestIQ Pro (web app) for landlords to manage properties, tenants, and payments
- Core value: reduce admin effort, improve visibility, and keep tenants informed
- Demo scope: property onboarding, tenant management, maintenance tracking, analytics, and tenant portal

Relevance of Topic

Why this topic fits the course & research objectives

Real-world problem

Rental admin is error-prone and time-consuming
Small landlords often use spreadsheets/WhatsApp
Visibility across properties is limited

Users

Landlord/manager: operations + finance overview
Tenant: communication + payments + requests
Prospective tenant: applications workflow

Course alignment

Information systems + UI/UX design
Database-backed CRUD + reporting
Security & user roles (landlord vs tenant)

Research angle

How digitising workflows improves timeliness
Reducing missed tasks via reminders & dashboards
Map + analytics to support decisions

Problem Statement

Clear problem + objectives

- Problem: Landlords struggle to track rental operations (tenants, maintenance, expenses, reconciliations) consistently across multiple properties.
- Objective 1: Provide a single dashboard to manage property and tenant data.
- Objective 2: Support operational tasks (maintenance, inspections, calendar) with clear status tracking.
- Objective 3: Improve financial visibility (rent roll, expenses, reconcile, tax outputs).
- Objective 4: Enable tenant self-service via a portal (requests, payments, communication).

Feasibility

Scope, constraints, and what we can deliver

What is feasible now

Core CRUD flows for properties/tenants
Photo upload + map pin for properties
Dashboards + analytics + reports
Tenant portal pages

Constraints

Accuracy depends on map data for reverse-geocode
Offline limited to cached UI where applicable
Browser geolocation permissions required

Data & security

Cloud Firestore for multi-tenant data
Firebase Auth for login/roles
Storage for property photos

Milestones

Prototype UI + data model
Integration (auth + database + storage)
Testing + demo preparation

Originality & Innovation

What makes NestIQ different

- Integrated landlord + tenant views in one lightweight web app.
- Map-based pinning of properties for portfolio visibility.
- Operational workflows alongside finance (maintenance, inspections, calendar, follow-ups).
- Command palette + fast navigation to reduce friction for frequent tasks.

Potential Impact

Who benefits and how

- Landlords: faster admin, fewer missed tasks, clearer financial picture.
- Tenants: simpler communication, better transparency and response times.
- Long-term: scalable foundation for automation (late fees, reminders, renewals).

UI Design (Mock-up)

Key screens and interaction flow

- Navigation: primary pages + "More" menu to reduce clutter.
- Add/Edit Property: photo upload + map pin + "use current location".
- Portfolio Map: visualize properties; click pins for quick details.
- Tenant Portal: tenant dashboard with requests and payment visibility.

Landlord Dashboard

Properties | Tenants | Analytics | More

- Portfolio summary cards
- Quick actions: Add Property / Add Tenant
- Upcoming renewals and inspections
- Maintenance tracker overview

Add/Edit Property (Modal)

Upload photo | Pin on map | Use current

- Photo picker + preview
- Map pin sets latitude/longitude
- Reverse-geocode fills address + city
- Drag pin to adjust location

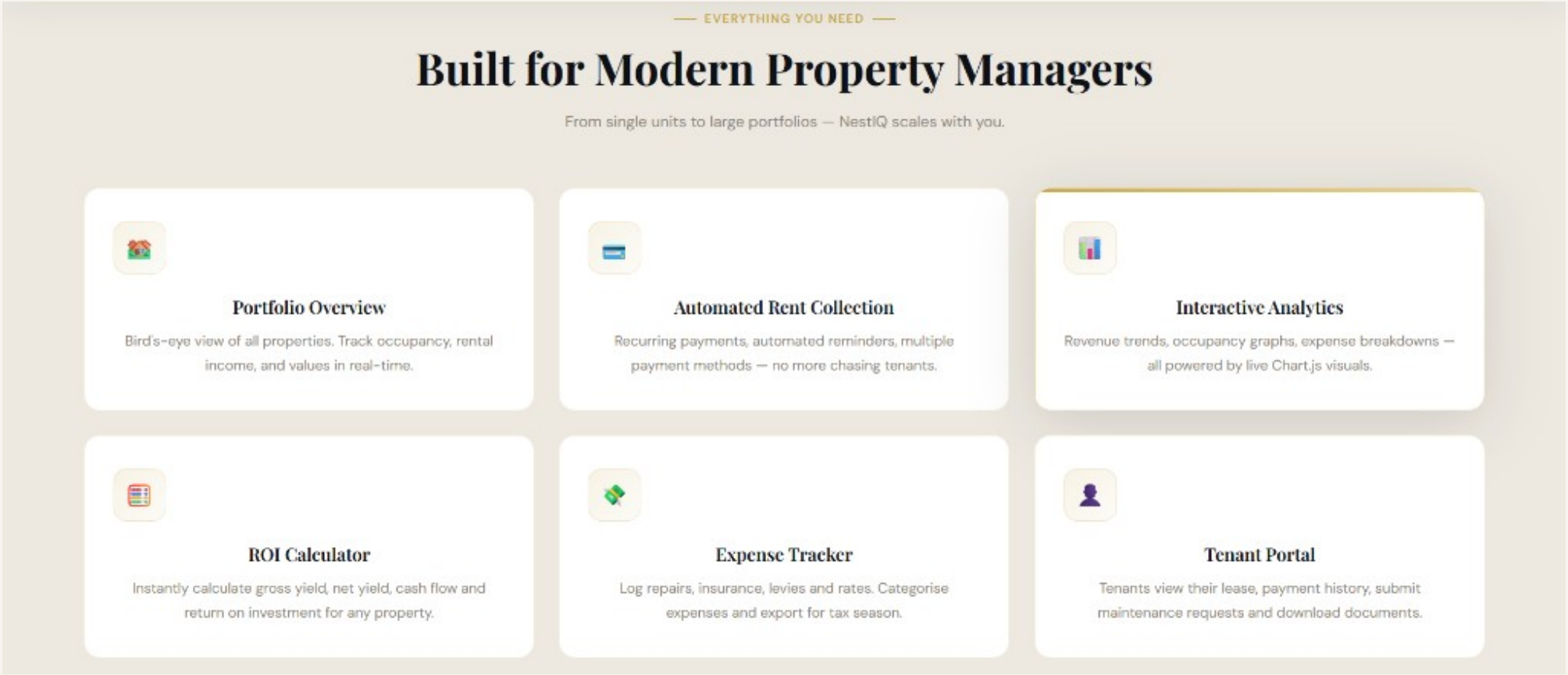
Tenant Portal

Dashboard | Payments | Requests

- Rent status and due dates
- Maintenance request form + history
- Notifications / follow-ups
- Contact / communication log

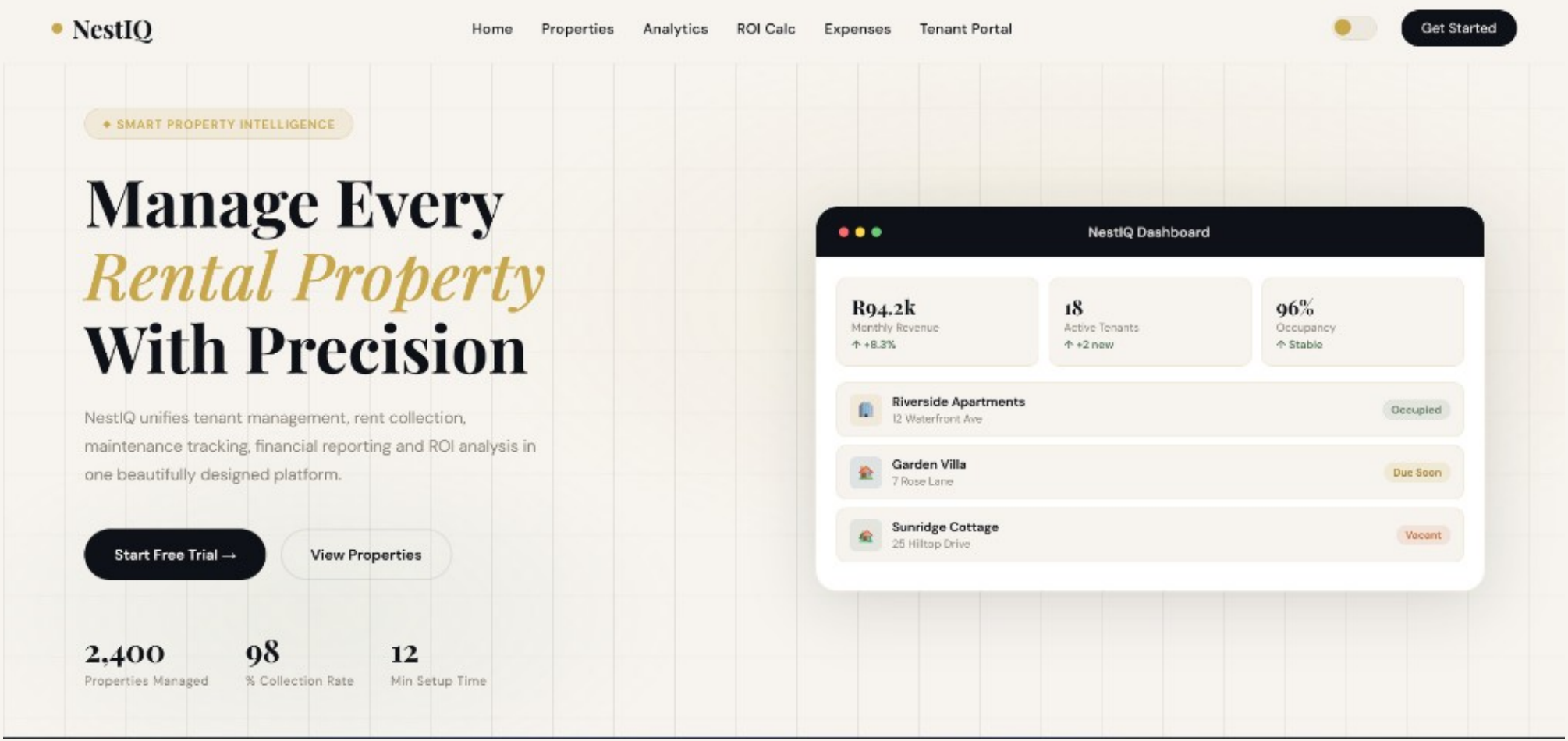
Live UI - Features section

Built for Modern Property Managers (product capabilities)



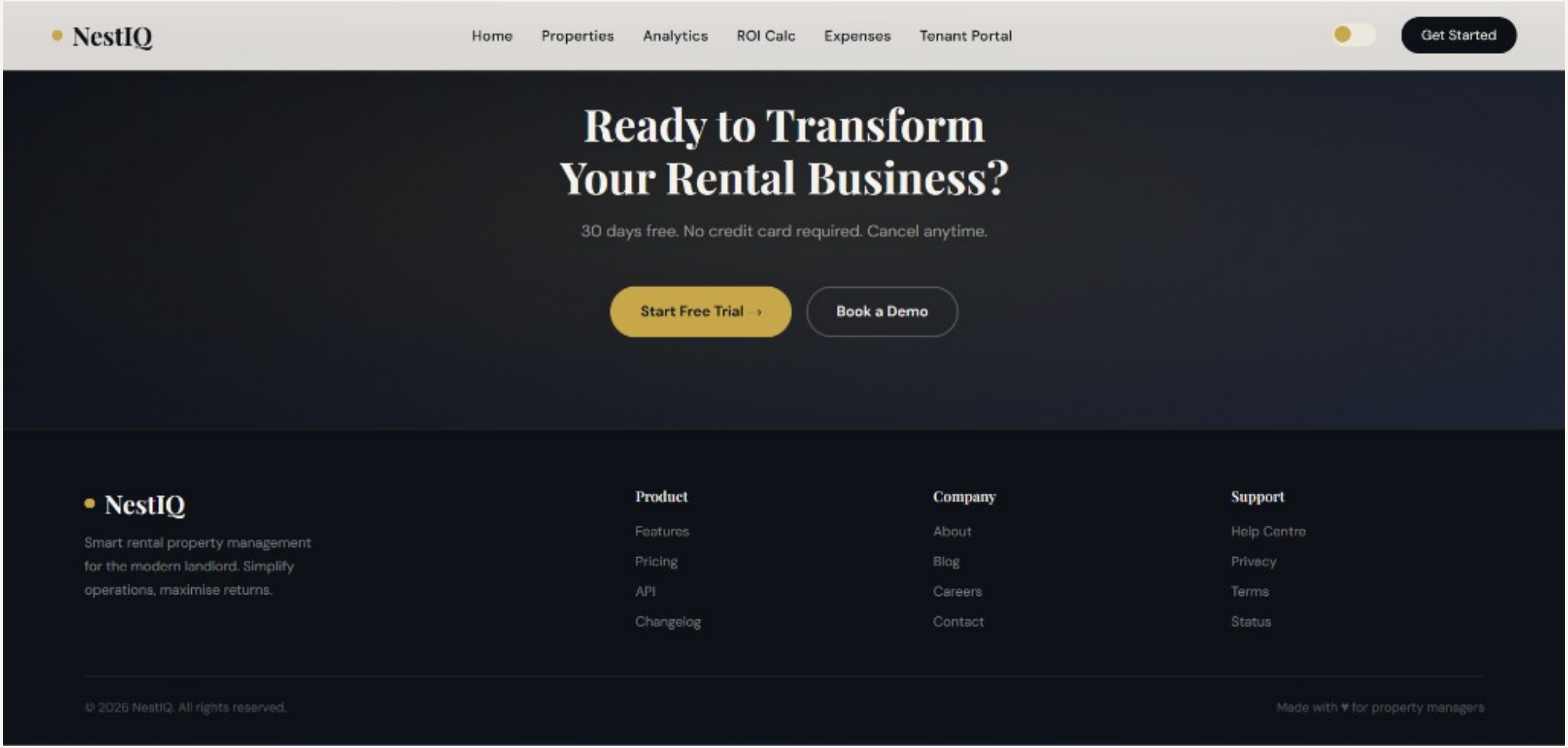
Live UI - Home (hero)

Landing hero + NestIQ dashboard preview



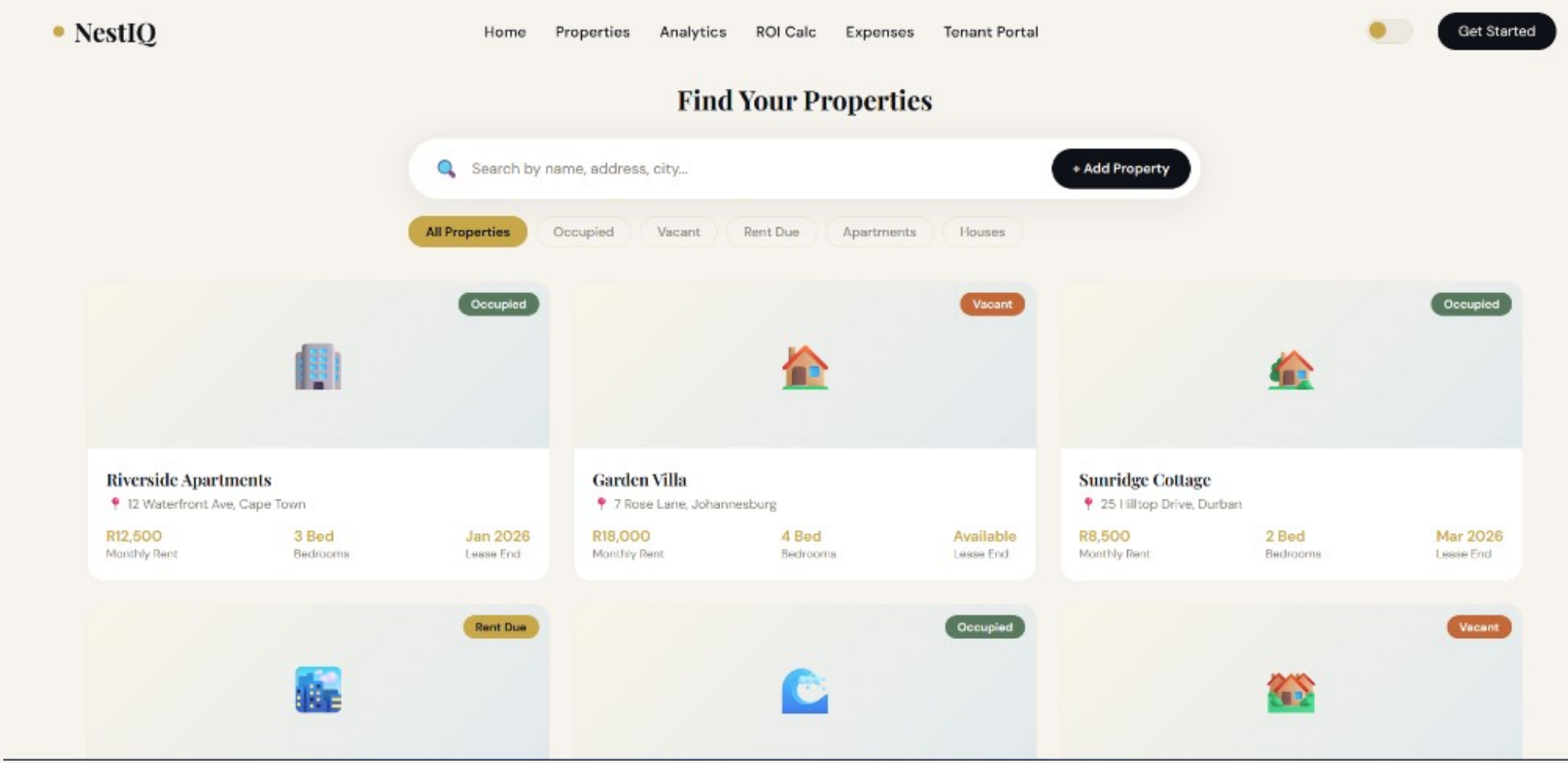
Live UI - Marketing CTA

Conversion section + footer (brand + next steps)



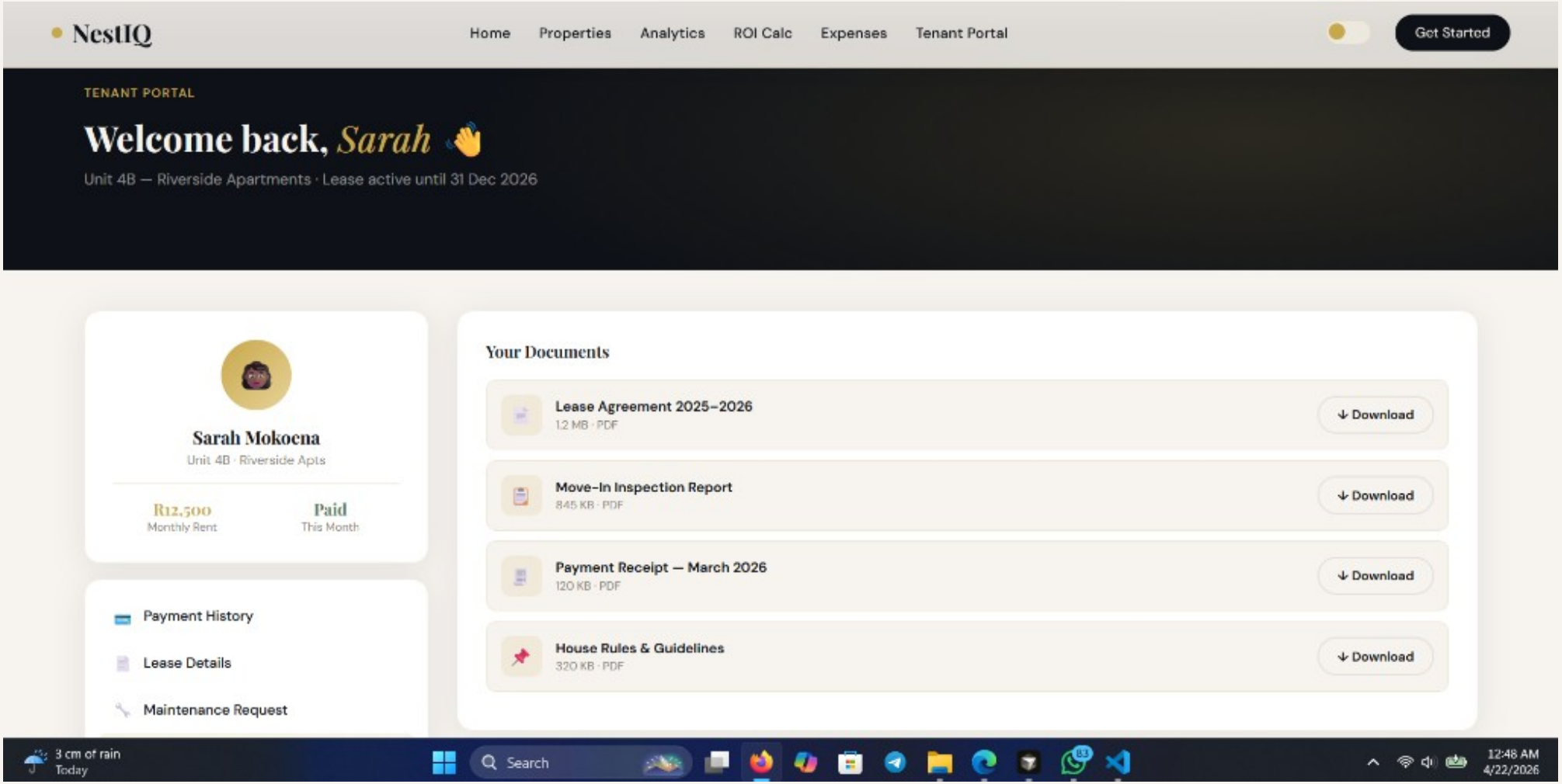
Live UI - Properties directory

Search, filters, and property cards



Live UI - Tenant portal

Tenant dashboard, profile, and documents



Project Plan, WBS & Tools

Breakdown of tasks + technologies

Work Breakdown Structure (WBS)

1. Requirements + rubric mapping
2. UI/UX mockups + navigation design
3. Data model (properties, tenants, expenses, maintenance)
4. Implementation (pages + modals + routing)
5. Integrations (Firebase Auth/Firestore/Storage)
6. Testing + demo + documentation

Tools / Technologies

Frontend: HTML, CSS, Vanilla JS
Maps: Leaflet + OpenStreetMap tiles
Reverse geocode: Nominatim (OSM)
Backend services: Firebase (Auth, Firestore, Storage)
Charts/Reports: Chart.js + jsPDF (where needed)

Collaboration

Task allocation by feature areas
Shared coding standards & review
Incremental testing and integration

Presentation & Teamwork

How the team will present

- Each member presents a section: problem -> design -> implementation -> demo -> plan.
- Live demo walkthrough with backup screenshots in case of connectivity issues.
- Shared Q&A: members support each other during questions.

Thank you

Questions & feedback

- Demo: Add property -> pin location -> view on map -> tenant portal preview
- We welcome feedback on UX, scope, and next features to prioritise.